

Trabajo de Fin de Máster

APRENDIZAJE MULTITAREA PARA DETECCIÓN DE CÁNCER EN MAMOGRAFÍAS

MULTI-TASK LEARNING FOR BREAST CANCER DETECTION IN MAMMOGRAPHIES

Autor: **Fernández Barrill, Íñigo**

Tutores: García Domínguez, Manuel

Inés Armas, Adrián

MÁSTER

**MÁSTER EN CIENCIA DE DATOS Y APRENDIZAJE
AUTOMÁTICO**

ESCUELA DE MÁSTER Y DOCTORADO



**UNIVERSIDAD
DE LA RIOJA**

AÑO ACADÉMICO: 2022/2023

Índice

1. Introducción	2
1.1. Contexto	2
1.2. Motivación	3
1.3. Objetivos	3
1.4. Librerías y herramientas	4
2. Marco teórico	5
3. Análisis de los datos	10
3.1. Conjunto de datos	10
3.2. Reducción de volumen	11
3.3. Balanceo de datos	12
3.3.1. Balanceo por pacientes	12
3.3.2. Balanceo por imágenes	12
3.4. Limpieza del conjunto de datos	13
3.5. División del conjunto de datos	13
4. Evaluación inicial del problema	14
4.1. Modelos para imagen	14
4.1.1. Resultados sobre el conjunto de datos	14
4.2. Modelos para datos	15
5. Aproximaciones Multi Tarea	19
5.1. Primera aproximación	19
5.2. Segunda aproximación	22
5.2.1. Descripción de la arquitectura Multi Tarea de la segunda aproximación	23
5.2.2. Resultados de la arquitectura Multi Tarea de la segunda aproximación	26
6. Conclusiones	28
7. Bibliografía y Referencias	29
8. Anexos	30
8.1. Descripción del Conjunto de Datos	30
8.2. Resultados sobre los distintos conjuntos	32
8.3. Ajuste de parámetros para los modelos de datos	33
8.4. Combinación Óptima de variables en el conjunto de datos	35
8.5. Modelo Multi Tarea, Aproximación 2 con función de pérdida única	38

1. Introducción

1.1. Contexto

El cáncer es una de las principales causas de muerte a nivel mundial, en 2020 se diagnosticaron aproximadamente 18,1 millones de casos, y las previsiones estiman un aumento en las próximas dos décadas hasta los 27 millones de casos. Del mismo modo, la mortalidad de esta afección en 2020 fue de casi 10 millones de personas y se estima que aumente hasta los 16,3 millones para el año 2040. En cuanto al porcentaje de pacientes que logran superar la enfermedad este es de un 55,3 % en hombres y un 61,7 % en mujeres.[10]

Se trata de una enfermedad en la que se desarrollan células con anomalías que se multiplican y destruyen el tejido corporal. Hay sobre 100 tipos distintos de cáncer, con distintos síntomas y gravedades. Las principales causas de desarrollo de cáncer son el consumo de tabaco y los hábitos poco saludables. Pese a que hay enfermedades peores, a nivel social el cáncer tiene una fama más temible, pues su tratamiento es difícil y puede dejar secuelas graves. De hecho, es de las pocas afecciones que tienen un día mundial dedicado a él (4 de Febrero) y a su tratamiento se le llama comúnmente "Lucha contra el cáncer". También es la enfermedad a nivel mundial con más dinero destinado a combatirla.

De todos los tipos de cáncer, el más común es el de mama, que se desarrolla en el revestimiento de los órganos mamarios, principalmente en los conductos que transportan la leche desde la mama hasta el pezón y de los órganos que se encargan de producir la leche, que se llaman lobulillos. Este tipo de cáncer afecta principalmente a mujeres (más del 99 % de los casos se producen en mujeres[2]). Este tipo concreto de cáncer, pese a ser muy común, tiene una tasa de mortalidad baja, sin embargo, sus consecuencias son notables. Además, a pesar de tener una tasa de mortalidad baja, en los países occidentales es la primera causa de muerte en mujeres, la quinta a nivel global[1][2].

En este trabajo se ha decidido centrarse en el cáncer de mama, pues su prevención y detección temprana reduce significativamente el riesgo que presenta esta enfermedad. Si se detecta en las etapas iniciales el tratamiento es más sencillo y se puede llegar a mitigar la enfermedad antes de que esta presente síntomas.

En medicina las técnicas más extendidas para la detección del cáncer de mama son las mamografías, el uso de ecografías y la resonancia magnética, siendo la más común la primera de ellas. Una mamografía es una imagen de la mama realizada mediante rayos X y ayuda a identificar posibles signos de cáncer de mama en sus etapas iniciales. También puede usarse para una vez diagnosticado el cáncer, estudiar mejor el caso en concreto. Dentro de una mamografía los principales signos que se buscan son zonas que presenten una densidad mayor a la vista en el resto del tejido. Esto en la propia imagen se aprecia en zonas más claras dentro de las mamas. Si bien todas estas técnicas aportan información visual, este trabajo busca complementar esa información con datos sobre el paciente y dar una respuesta con más contexto. También, el proceso de análisis de estas imágenes se hace de forma manual y requiere que haya un médico disponible. Es por eso por lo que puede acarrear retrasos, y

un modelo basado en Inteligencia Artificial puede ayudar a solventar estos problemas y a acelerar el proceso.

1.2. Motivación

En los últimos años, con el avance de la Inteligencia Artificial, se han desarrollado muchas soluciones que permiten realizar procesos costosos y repetitivos de forma más sencilla. Algunos ejemplos de estos trabajos pueden ser el análisis de documentos mediante Reconocimiento Óptico de Caracteres, el uso de ChatBots para la atención al cliente o la clasificación de correos electrónicos mediante bolsas de palabras. El campo que más se está beneficiando actualmente de este desarrollo es el industrial, donde estas soluciones ayudan a optimizar la cadena de procesos.

Concretamente, en el ámbito de la medicina, el Aprendizaje Automático también está cada vez más extendido y ha abierto nuevas vías en la investigación de patógenos. Técnicas basadas en este campo se utilizan sobretodo en el diagnóstico (esto engloba la detección y la predicción), ya que un modelo tiene una mayor capacidad para manejar datos y es más eficiente, rápido y preciso a la hora de detectar patologías que un humano. De todas formas, esta tecnología precisa de supervisión humana, y más que un reemplazo, es siempre una herramienta que complementa y mejora el trabajo de las personas.

Es por eso, por lo que una herramienta de este tipo puede resultar muy útil en el diagnóstico de una enfermedad como puede ser el cáncer, cuyo factor más determinante para bajar la mortalidad es una detección a tiempo.

En el campo del Aprendizaje Automático cada vez se utilizan más las soluciones basadas en modelos Multi Tarea. Esto consiste en entrenar un modelo para que realice varias tareas relacionadas entre ellas de manera simultánea. Al enfocar el aprendizaje de esta manera, el modelo aprende las características comunes a todas las tareas, lo que resulta en un mejor rendimiento en comparación a la realización de estas tareas por separado. También, este procedimiento ayuda a la generalización del modelo y evita el sobreaprendizaje. En el caso concreto de este trabajo, como hasta ahora el cáncer de mama se diagnostica únicamente con la información que dan las imágenes (en este caso, las mamografías), al poder usar también la información asociada a los datos biológicos (como por ejemplo, la edad) usando un modelo Multi Tarea, que permite utilizar toda esa información, se puede mejorar el diagnóstico y acelerarlo.

1.3. Objetivos

Ya que el cáncer de mama es un gran problema, es muy importante que se estudien y se mejoren los procedimientos para su prevención, detección y tratamiento. También, teniendo en cuenta la situación actual en los hospitales, es necesario que se ayude al personal sanitario a agilizar la atención y mejorar su calidad.

Hay que tener en cuenta que debe haber un equilibrio a la hora de realizar el diagnóstico, pues si un caso positivo no se diagnostica, las consecuencias son graves, desde la muerte de la persona hasta un tratamiento más complicado al haber detectado la enfermedad en un estado más avanzado. En caso contrario, si se emite un falso positivo, también puede ser muy perjudicial, pues los tratamientos pueden ser caros e invasivos, además de las posibles secuelas que éstos tratamientos puedan dejar en la persona.

Por ello, el objetivo de este trabajo es estudiar las técnicas de Aprendizaje Multi Tarea para aplicarlos a la detección de cáncer en mamografías. Dicho de otra manera, el objetivo es realizar una clasificación para distinguir aquellas instancias que presentan cáncer de mama de las que no. Para ello, se van a estudiar diferentes modelos de Aprendizaje Profundo y comparar los resultados de esos mismos modelos con sus respectivas implementaciones en una aproximación Multi Tarea, para ver si se pueden obtener beneficios al usar este tipo de modelos.

1.4. Librerías y herramientas

Este trabajo se ha realizado en el lenguaje de programación *Python*, que es el más extendido para tareas relacionadas con el Aprendizaje Automático y cuenta con un amplio abanico de librerías especializadas en este campo, concretamente, este trabajo se va a apoyar en las siguientes librerías:

- *FastAI*[3]: Orientada a Aprendizaje Automático de alto nivel basada en *PyTorch*[12]. Es muy utilizada debido a su sencillez y potencia y a que permite realizar todas las tareas necesarias para crear y entrenar los modelos.
- *Timm*[14]: Proporciona una colección de distintos arquitecturas y modelos preentrenados en *PyTorch*[12], para tareas relacionadas principalmente con la visión por computador.
- *NumPy*[9]: Es la librería más utilizada en *Python*, que se basa en el tratamiento de objetos y datos numéricos. Su principal estructura es el *Ndarray*, que a grandes rasgos, es un vector multidimensional.
- *Pandas*[11]: Está orientada al análisis de datos y proporciona estructuras de datos (como el *DataFrame*) y funciones para tratarlos. Un *DataFrame* se compone de un conjunto de *Series*, y se puede comparar con una tabla de datos.
- *Scikit-learn*[13]: Ofrece métricas y herramientas para evaluar los modelos de Aprendizaje Automático.
- *PyTorch*[12]: Está orientada al Aprendizaje Automático y facilita las labores de proceso al implementar lo que se conoce como *Tensor*, que es una estructura multidimensional cuya característica principal es que dentro de una misma dimensión los tensores no tienen por qué tener el mismo tamaño, permitiendo así, entradas de datos irregulares. Está basada en la biblioteca *Torch*[15] de *C*, que ya no se desarrolla, pero sigue siendo importante.

2. Marco teórico

Antes de comenzar a profundizar en el trabajo, es conveniente entender algunos de los conceptos que se tratan en él.

La tarea que se lleva a cabo en este trabajo es la clasificación, que consiste en dado un conjunto de datos y un conjunto de etiquetas, asignar a cada dato la etiqueta correcta. En este caso concreto, dos etiquetas o clases (esto se conoce como clasificación binaria), la clase positiva, con las instancias que estén etiquetadas con cáncer de mama, y la clase negativa, con aquellas etiquetadas sin cáncer de mama. Para realizar esta clasificación a lo largo de los años se han tratado diferentes formas, sin embargo, con el auge de las técnicas de Aprendizaje Profundo, estos problemas se están resolviendo con este tipo de modelos dando buen rendimiento. En la siguiente figura se puede ver un ejemplo de clasificación binaria, para que se entienda mejor.

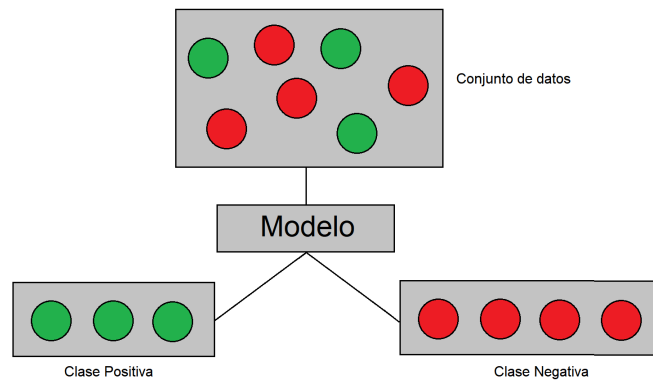


Figura 1: Ejemplo de clasificación en dos clases. Siendo la clase positiva aquella con las instancias etiquetadas con el color verde y la clase negativa aquella con las instancias etiquetadas con el color rojo.

Liendo por partes, la Inteligencia Artificial es un campo que busca reproducir aspectos del pensamiento humano mediante el uso de la tecnología. La idea principal es procesar información, aprender de ella y utilizar lo aprendido para realizar tareas. Estas tareas pueden ser el razonamiento, el aprendizaje, la percepción...

El Aprendizaje Automático es una rama de la Inteligencia Artificial, centrada en el desarrollo de sistemas y algoritmos que sean capaces de aprender y tomar decisiones sin ser programados de manera explícita para ello. Es decir, se programan para que aprendan a realizar la tarea, no para que la realicen directamente. Las técnicas que se utilizan permiten que los modelos basados en ellas aprendan a medida que reciben datos.

Dentro del Aprendizaje Automático está el Aprendizaje Profundo. Principalmente se basa en Redes Neuronales profundas para aprender y extraer características comunes de los datos.

Su nombre, profundo, viene debido a la cantidad de capas que tienen las Redes Neuronales. Está inspirado en la estructura y funcionamiento del cerebro humano, en el que cada zona del cerebro se encarga de una tarea, tal y como cada capa dentro de la Red Neuronal se encarga de una tarea o una parte concreta.

En Aprendizaje Profundo, se le llama modelo a toda aquella arquitectura que cuenta con la capacidad de extraer información de los datos. Estos datos pueden ser tablas o imágenes. En el caso de las imágenes, los modelos más comunes en las labores de clasificación se basan en arquitecturas de *CNN* (Redes Neuronales Convolucionales), *RNN* (Redes Neuronales Recurrentes), *GAN* (Redes Adversarias Generativas) y Transformadores.

Las *CNN* son un tipo de Red Neuronal profunda diseñada para procesar datos con una estructura en forma de cuadrícula, por lo que funciona muy bien con imágenes y con audio. De forma resumida, funciona reduciendo la dimensión de los datos para quedarse con las características importantes y poder clasificar mejor.

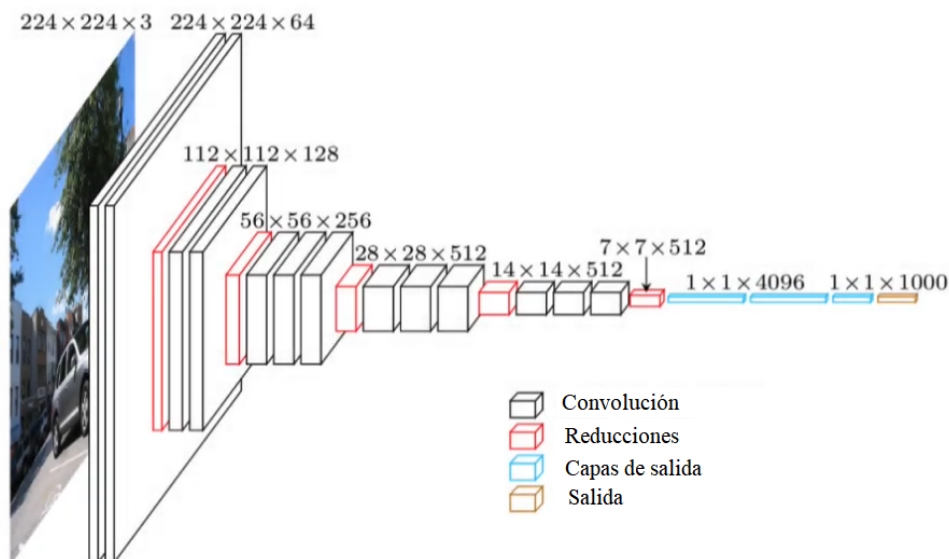


Figura 2: Visualización de una arquitectura de Red Neuronal Convolutional.

Las *RNN* son un tipo de Red Neuronal diseñada para procesar datos que siguen una distribución secuencial, por lo que funciona muy bien con datos temporales. El nombre de Recurrente viene dado porque tiene conexiones hacia atrás en las capas posteriores (llamadas conexiones de realimentación), creando así algo parecido a una "memoria" que pueda aprender patrones en secuencias de datos. Este tipo de arquitectura funciona muy bien para la clasificación de textos.

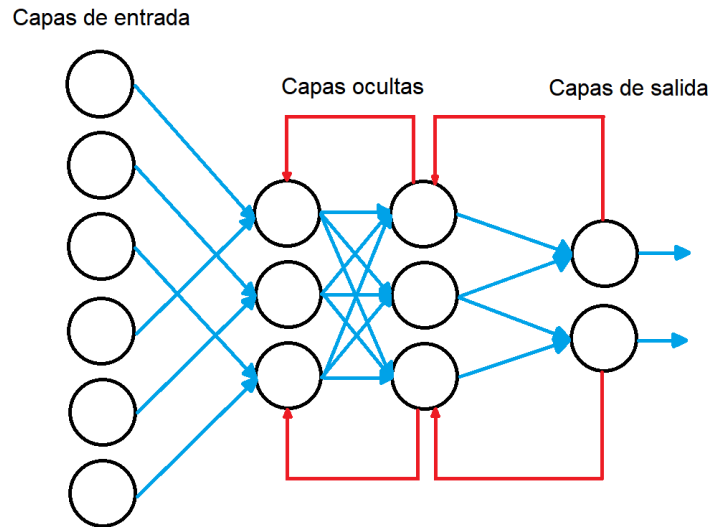


Figura 3: Visualización de una arquitectura de Red Neuronal Recurrente.

Las *GAN* son un tipo de Red Neuronal que en el que hay dos redes enfrentadas, una que genera datos y otra que los discrimina. De esta forma lo que se consigue es que la una mejore a la otra, la red generadora va a intentar hacer mejores datos para .engañar.^a la red discriminadora y ésta otra va a aprender a distinguir mejor los datos generados por la primera red de los datos reales. Este tipo de arquitectura se utilizan en tareas relacionadas principalmente con imágenes.

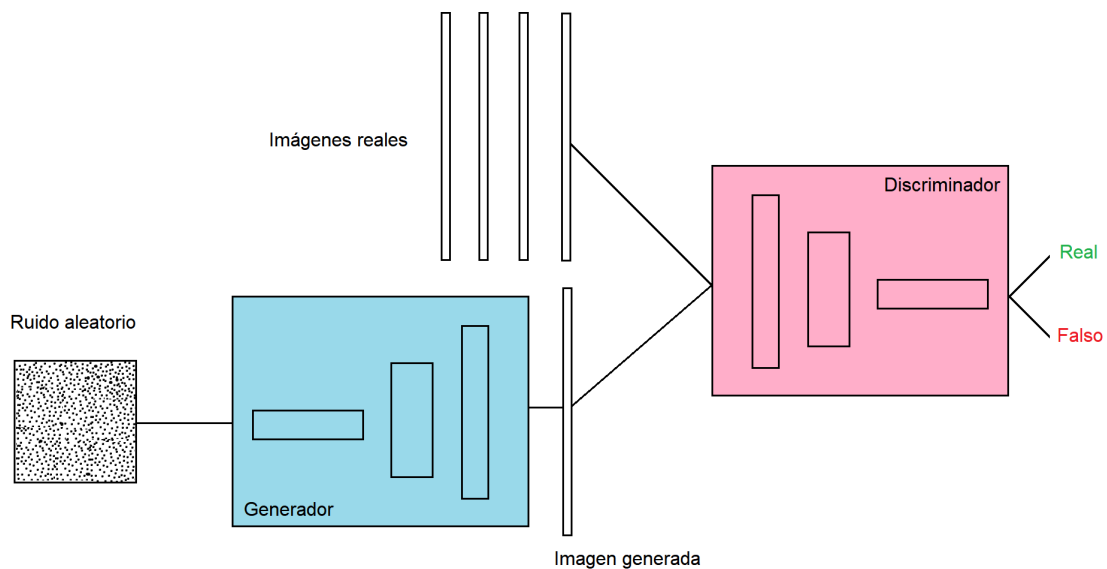


Figura 4: Visualización de una arquitectura de Red Adversaria Generativa.

Los Transformadores son un tipo de Red Neuronal diseñado con dos componentes, codificadores y decodificadores. El codificador procesa la entrada y el decodificador genera la salida con los datos procesados. La característica que los diferencia del resto de Redes Neuronales es la capacidad de capturar información contextual en paralelo, en vez de hacerlo de manera secuencial. Esto los convierte en arquitecturas más eficientes en términos de tiempo de entrenamiento y procesado.

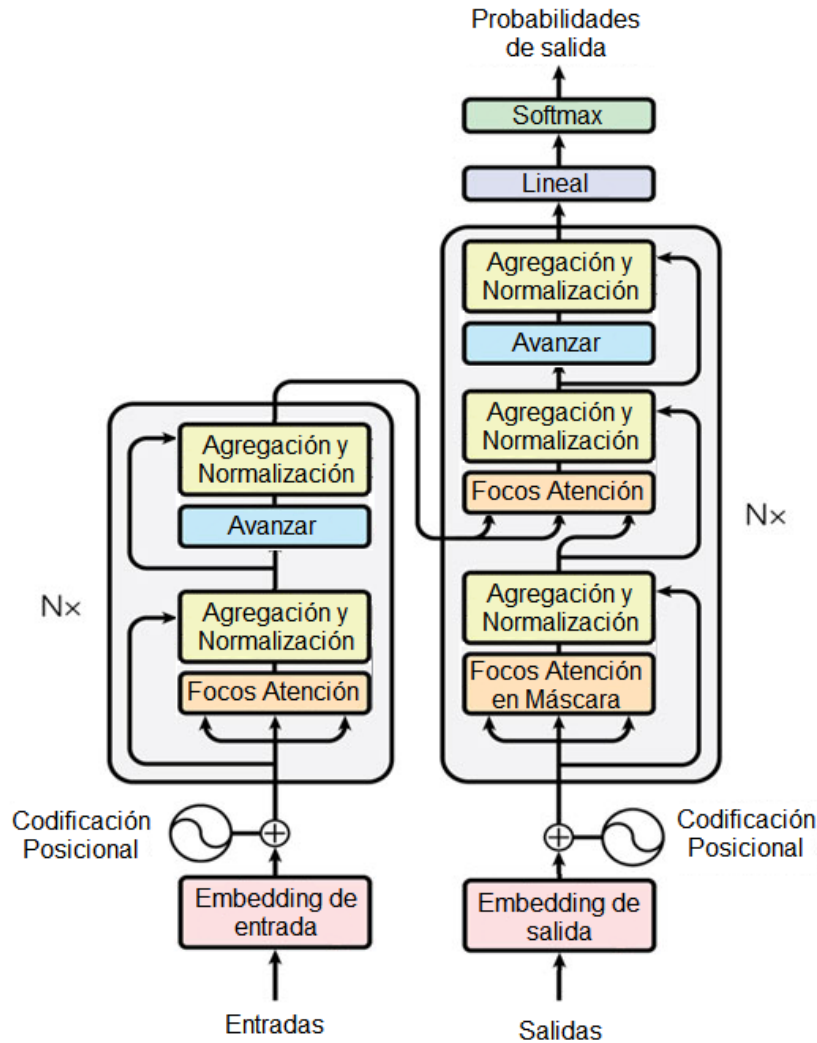


Figura 5: Visualización de una arquitectura de Transformador.

De todas estas arquitecturas, las que realizan tareas de clasificación son las *CNN* y los Transformadores, por tanto, son las que se van a utilizar en este trabajo.

Pese a no ser la aproximación que más se está utilizando en los últimos años, el Aprendizaje Multi Tarea va un paso más allá. Si bien, los modelos tradicionales tratan una entrada

para dar una salida, es decir, dan una respuesta a un conjunto de datos, los modelos Multi Tarea realizan, tal y como indica su nombre, varias tareas sobre uno o varios conjuntos de datos. De esta forma, se puede extraer aún más información y al aprender características comunes a todas las tareas se puede dar una respuesta más precisa. También este tipo de modelos tienden a sobreajustar menos. Dado el problema que presenta este trabajo, este tipo de modelos son los más adecuados, pues se busca tanto realizar varias predicciones distintas, como extraer información de varios tipos de datos a la vez. Estas dos aproximaciones a los modelos Multi Tarea, se explican en profundidad más adelante en su respectivo apartado.

3. Análisis de los datos

En este apartado se explica el conjunto de datos que se va a utilizar en este trabajo. También se detallan los tratamientos y transformaciones realizadas sobre el conjunto, así como su limpieza.

3.1. Conjunto de datos

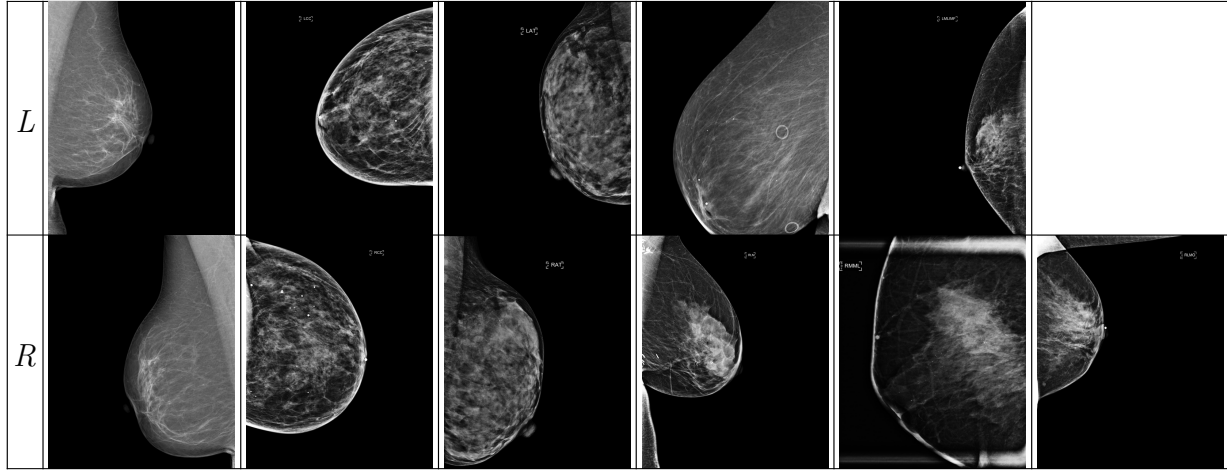
El conjunto de datos es el *RSNA Screening Mammography Breast Cancer Detection*[8], que cuenta con imágenes de mamografías y datos estructurados sobre cada paciente. Tiene un dos carpetas, *test_images* y *train_images*, cada una un número distinto de carpetas, teniendo cada carpeta las mamografías de cada paciente. En la carpeta *test_images* sólo hay una carpeta con cuatro imágenes, y en *train_images* hay 11.913 carpetas.

También, hay archivos en formato *.csv* (*Comma-Separated Values*) con los datos de los pacientes y de las imágenes, ver Cuadro 1. Estos archivos son *train.csv*, con el conjunto de Entrenamiento (54.710 imágenes), *test.csv*, con el conjunto de Test (4 imágenes) y *sample_submission.csv*, con una muestra para la comprobación. Este último archivo se va a obviar, pues es un fichero propio de la competición de *Kaggle*. En estos archivos cada fila representa una imagen. Un análisis más en profundidad del conjunto de datos y su distribución se puede consultar en el Anexo 8.1.

<i>site_id</i>	<i>patient_id</i>	<i>image_id</i>	<i>laterality</i>	<i>view</i>	<i>age</i>	<i>cancer</i>	<i>biopsy</i>	<i>invasive</i>	<i>BIRADS</i>	<i>implant</i>	<i>density</i>	<i>machine_id</i>	<i>d_n_c</i>
1	9973	1729524723	R	MLO	43.0	0	0	0	1.0	0	C	49	False

Cuadro 1: Ejemplo de instancia.

El conjunto total son 54.710 imágenes en formato *.DICOM*. Este formato (*Digital Imaging and Communication In Medicine*) es el estándar que se utiliza en medicina para la transmisión de imágenes y datos. En el caso de este conjunto, las imágenes son las mamografías y los datos asociados a ellas son el identificador del paciente, el tipo de imagen, el tamaño de la imagen, la fecha de la imagen y muchas otras especificaciones técnicas sobre la imagen. En el Cuadro 2 hay un ejemplo de cómo son las mamografías sobre las diferentes vistas desde las que son tomadas.



Cuadro 2: Mamografías de ejemplo, según sus vistas.

En el conjunto sólo hay una imagen con la vista *LMO*, y es del pecho derecho, por lo que no hay imagen en esta vista para la mama izquierda en el Cuadro 2. Se puede ver que son imágenes en escala de grises en donde el fondo aparece en un tono oscuro mientras que la mama se ve porque se resalta en más claro.

Las imágenes están estructuradas de forma que por cada paciente hay cuatro imágenes, dos por cada pecho (este es el estándar, pero hay algunos pacientes que tienen un número distinto de imágenes). Cada imagen tiene como nombre de archivo su identificador en la tabla de datos y están dentro de una carpeta con el nombre del identificador del paciente.

Como este trabajo no está orientado a participar en la competición de *Kaggle*, no se va a tomar en cuenta el archivo *sample_submission.csv*, y se va a obviar el archivo *test.csv*, pues sólo cuenta con 4 entradas, y prácticamente el total del conjunto de datos están en *train.csv* (quitando las 4 imágenes del conjunto de test, éste tiene 54.706 imágenes, las de la carpeta *train_images*), y éste será el conjunto que se usará de aquí en adelante.

3.2. Reducción de volumen

El primer problema que plantea el conjunto de datos es su gran tamaño. En total pesa 314,72 GB y esto lo hace inviable para trabajar con él con las herramientas con las que se dispone. Todo este volumen viene por parte de las imágenes, que presentan dos características que las hacen especialmente pesadas. La primera es el propio formato, como se ha mencionado antes, el formato *.DICOM* adjunta a las imágenes cierta información, de la que realmente sólo interesa el identificador del paciente. Sin embargo, esta información realmente tampoco es necesaria, pues únicamente con el nombre de la imagen es suficiente para ubicarla dentro del *.csv*. Por lo tanto, no es necesario que la imagen esté en formato *.DICOM*, así que se transforma a *.jpg*. Esta transformación reduce el tamaño total del conjunto de datos a un tercio del volumen original.

En segundo lugar, las imágenes son de tamaño 2082x2776 e incluso 4915x5355. Los modelos de aprendizaje profundo suelen utilizar tamaños de imagen más pequeños, por lo que

el segundo paso es reducir la dimensión de las imágenes. Manteniendo las proporciones generales de las imágenes, se han reducido a un tamaño de 512x640 píxeles. Realizando estas transformaciones el tamaño total del conjunto es de 5,72 GB, lo que lo hace mucho más manejable.

3.3. Balanceo de datos

Como se puede ver en el Anexo 8.1, para la variable objetivo del conjunto de datos, que es la variable ‘*cancer*’, no hay una distribución equilibrada (solamente el 2 % de las imágenes está catalogada como caso positivo en cáncer). Esto indica que es necesario balancear los datos para igualar el número de instancias en ambas clases.

Para balancear los datos las dos técnicas más extendidas son el oversampling y el undersampling. La primera consiste en aumentar de manera artificial el conjunto de la clase minoritaria para que haya un número parecido de instancias en ambas clases. La segunda, consiste en disminuir el tamaño de la clase mayoritaria, normalmente se hace mediante la selección aleatoria de instancias.

Hay dos opciones sobre cómo realizar este balanceo, ambas siguiendo la técnica de undersampling, porque, al ser un tema tan específico y en el que el fallo es tan castigado, crear datos sintéticos puede ser peligroso, por lo que es mejor usar los datos reales que ya se tienen. Las dos opciones son balancear por pacientes o por imágenes. Esto se explica a continuación.

3.3.1. Balanceo por pacientes

En el conjunto de datos hay 486 pacientes que han dado positivo en alguna de sus imágenes. Aplicando esta técnica de balanceo, el resultado final es un conjunto de datos reducido, con 972 pacientes.

El conjunto de las imágenes queda reducido también a 4.584 imágenes, de las cuales 1.158 (25 %) tienen cáncer y 3.426 (75 %) no. Esta opción pese a ser una aproximación más realista, sigue estando desbalanceada, ya que los pacientes que han dado negativo tienen todas sus fotos con resultado negativo, mientras que los pacientes positivos tienen la mitad de sus fotos negativas y la mitad positivas (ya que lo normal es presentar cáncer de mama en un pecho y no en los dos).

3.3.2. Balanceo por imágenes

En el conjunto de datos hay 1.158 imágenes con cáncer. Tras aplicar el balanceo, quedan un total de 2.305 imágenes, las 1.158 positivas y 1.147 negativas. En este balanceo los pacientes con imágenes negativas tendrán generalmente una única imagen, mientras que aquellos con cáncer tendrán dos (las dos vistas estándar por cada pecho). Esto se aleja del caso real, pero se acerca más a los conjuntos de entrenamiento de modelos de Aprendizaje Profun-

do, que cuentan con conjuntos con balanceos cercanos al 50 % porque son los que mejores resultados dan. Así que este es el conjunto que se va a utilizar de aquí en adelante.

3.4. Limpieza del conjunto de datos

Con el conjunto de datos balanceado, se tienen valores faltantes (*NaN*) en la variable ‘*density*’, que es una variable categórica. Como este es un conjunto de datos real, en medicina los casos similares suelen tener características también parecidas. Es por eso por lo que los valores faltantes se van a imputar mediante el uso de *KNN*, que clasifica las instancias en las 4 categorías de densidad del conjunto teniendo en cuenta los casos que más se le parecen. De las 2.305 instancias del conjunto balanceado por imágenes, hay 1.019 valores faltantes (un 44 % del total), que, tras imputarlos han sido categorizados en su mayoría en la clase *B*, quedando finalmente de la siguiente manera:

- *A*: 204 (9 %).
- *B*: 1323 (57 %).
- *C*: 708 (31 %).
- *D*: 70 (3 %).

3.5. División del conjunto de datos

Finalmente, para todas las pruebas con modelos que se van a realizar, se va a separar el conjunto en dos, uno para entrenar los modelos y otro para probar los resultados. El ratio que más se suele utilizar y que mejores resultados da es de un 80 % del total para el conjunto de entrenamiento y un 20 % para el conjunto de validación. Esta separación se hace mediante la función *RandomSplitter*, que separa las instancias de manera aleatoria. Esta será la separación de aquí en adelante para los modelos de imagen.

Para los modelos de datos, se va a utilizar una separación diferente. Un 70 % para el conjunto de entrenamiento, un 10 % para el conjunto de validación y un 20 % para el conjunto de prueba, mediante la función *train_test_split*. En este caso se evaluará el mejor modelo y los mejores parámetros con el conjunto de validación y, una vez hecho esto, reentrenar el modelo juntando el conjunto de entrenamiento y validación para dar los resultados finales sobre el conjunto de prueba. Esta división en tres partes sirve para ajustar mejor los parámetros del modelo.

4. Evaluación inicial del problema

La idea inicial es entrenar dos modelos distintos, uno para obtener información de las mamografías y otro para predecir el cáncer con los datos asociados a las mismas de los *.csv*. Una vez se tengan ambos modelos, se buscará una aproximación mediante el uso de una arquitectura Multi Tarea, y se comprobará si los resultados mejoran o empeoran al usar ese tipo de modelo.

4.1. Modelos para imagen

Los modelos elegidos son los siguientes:

- *ResNet 50*[4], que es un modelo basado en arquitectura de Red Neuronal.
- *HrNet w44*[17], que es un modelo basado en arquitectura de Red Neuronal.
- *EficientNet B3*[5], que es un modelo basado en arquitectura de Red Neuronal Convocional.
- *ConvNext*[6], que es un modelo basado en arquitectura Transformer.
- *Deit3 base patch 16 384*[16], que es un modelo basado en arquitectura Transformer.

4.1.1. Resultados sobre el conjunto de datos

Para la evaluación de los distintos modelos sobre las imágenes, se ha probado con el conjunto balanceado por imágenes y los modelos mencionados, modificando el factor de aprendizaje, los callbacks a la hora de entrenar y el número de épocas que se ejecutan. La comparación de resultados se realiza mediante tablas, en las que se muestran los mejores valores de cada prueba junto con los minutos tardados en llegar a ella y las épocas transcurridas desde el inicio. También, los resultados se evalúan respecto a la variable ‘*cancer*’, que es la variable objetivo del conjunto de datos. Las métricas usadas son las más comunes: Accuracy, Precision, Recall y F-Score, siendo esta última la más importante debido al tipo de problema, pues es la más usada en el ámbito del diagnóstico médico y es peligroso fallar tanto en casos positivos como negativos.

La máquina que se ha utilizado para entrenar son las de *Google Colab*. Esta máquina cuenta con un procesador *Intel Xeon CPU* con una frecuencia de 2.20 GHz, 13 GB de memoria RAM, un acelerador *Tesla K80* y 12 GB GDDR5 de VRAM. Se entrena durante 30 épocas y con un ratio de aprendizaje de 1e-3, salvo *Resnet50*, que como control se le aumenta el número de épocas a 50, en caso de que se produzca sobreaprendizaje. Si se quieren ver los resultados con los otros conjuntos (con los datos sin balancear y los datos balanceados por pacientes) se pueden ver en el Anexo 8.2.

	<i>Resnet50</i>	<i>EfficientNet</i>	<i>ConvNext</i>	<i>Hrnet</i>	<i>Deit</i>
Accuracy	0,6610	0,5254	0,5876	0,5650	0,6271
Precision	0,5807	0,4444	0,5076	0,4918	0,5584
Recall	0,7200	0,4800	0,8933	0,8000	0,5733
F-Score	0,6429	0,4615	0,6473	0,6091	0,5658
Tiempo (Minutos)	13	0,25	8	11	6
Época	38	1	8	10	20

Cuadro 3: Resultados sobre el conjunto de imágenes balanceado por imágenes, con el mejor resultado resaltado en **negrita** (de aquí en adelante se resalta de esta manera el mejor resultado de cada Cuadro).

En el Cuadro 3, el mejor modelo ha sido *ConvNext*, que ha logrado los mejores resultados en el menor número de épocas (Sin tener en cuenta a *EfficientNet*), con un F-Score de 0,6473.

Estas tablas pueden servir para hacerse una idea general del rendimiento de los modelos, pero normalmente, se suelen aplicar callbacks de *Early Stopping*, que si no encuentran una mejora de rendimiento entre las épocas, detienen el entrenamiento. Esto previene el sobreajuste y optimiza el tiempo de entrenamiento. Los resultados son los siguientes.

	<i>Resnet50</i>	<i>EfficientNet</i>	<i>ConvNext</i>	<i>Hrnet</i>	<i>Deit</i>
Accuracy	0,5380	0,4642	0,6117	0,5835	0,6486
Precision	0,5309	0,3478	0,5916	0,60000	0,6486
Recall	0,9667	0,0333	0,8208	0,6000	0,6792
F-Score	0,6854	0,0608	0,6876	0,6000	0,6680
Tiempo (Minutos)	1	1	16	12	9
Época	1	1	8	4	8

Cuadro 4: Resultados sobre el conjunto de imágenes balanceado por imágenes, aplicando *Early Stopping*.

En el Cuadro 4 se ha conseguido un F1-Score de 0,6876, mejor aún que en el Cuadro 3, con 0,6473. De esta manera se han conseguido los mejores resultados con el modelo *ConvNext* y al usar *Early Stopping* además se consigue evitar el sobreajuste.

4.2. Modelos para datos

Siguiendo con el conjunto balanceado por imágenes, se utilizan los datos asociados a las imágenes y se preparan para entrenar los modelos, que necesitan dos conjuntos, X (con las variables con las que calcular la variable objetivo) e y (con la variable objetivo). Teniendo en cuenta que en el conjunto de datos hay variables que se aportan una vez se ha detectado el cáncer, no serán tomadas en cuenta. Los conjuntos quedan de la siguiente manera:

- X :
 - *'site_id'*.
 - *'patient_id'*.
 - *'image_id'*.
 - *'laterality'*.
 - *'view'*.
 - *'age'*.
 - *'implant'*.
 - *'density'*.
 - *'machine_id'*.
- y :
 - *'cancer'*.

Y se distribuye así:

- 70 % para el conjunto de entrenamiento.
- 10 % para el conjunto de validación.
- 20 % para el conjunto de prueba.

Las variables desechadas en este conjunto son *'invasive'*, *'biopsy'*, *'BIRADS'* y *'difficult_negative_case'*, que son aquellas que en primera instancia sólo aparecían en el conjunto de Entrenamiento.

Para evaluar los datos, los modelos seleccionados son los siguientes:

- *Árbol C4.5*.
- *Árbol CART*.
- *Random Forest*.
- *Regresión Logística*.
- *Red Neuronal*.
- *SVM*.

Los modelos obtienen los siguientes resultados (Los detalles sobre los parámetros de los modelos están en el anexo 8.3):

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6000	0,6565	0,7000	0,5043	0,4957	0,4957
F1-Score clase negativa	0,59	0,66	0,69	0,67	0,00	0,00
F1-Score clase positiva	0,61	0,66	0,71	0,00	0,66	0,66

Cuadro 5: Resultados sobre el conjunto de validación con todas las variables.

En el Cuadro 5 el mejor modelo ha sido el *Random Forest*, con un F-Score de 0,70 pero se puede ver que en otros modelos hay clases que no obtienen representación (En los modelos de *Red Neuronal* y *SVM*). Esto puede ser debido a que los datos no están normalizados y se esté dando más peso a las variables con valores más altos. Si se normalizan los datos y se repite la evaluación de los modelos se obtienen los siguientes resultados:

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,5826	0,6304	0,6696	0,5043	0,6304	0,6304
F1-Score clase negativa	0,59	0,64	0,67	0,58	0,62	0,62
F1-Score clase positiva	0,58	0,63	0,67	0,63	0,64	0,64

Cuadro 6: Resultados sobre el conjunto de validación normalizado con todas las variables.

En el Cuadro 6 el modelo *Random Forest* ha vuelto a dar los mejores resultados, con un F-Score de 0,67, aunque no tan buenos como los obtenidos sin normalizar del Cuadro 5, que obtenía un F-Score de 0,70. Los resultados en los modelos que antes fallaban más han mejorado mucho, mientras que los que antes funcionaban bien, ahora funcionan ligeramente peor. Esto puede ser porque las variables con los valores más elevados sean las que más importancia tengan a la hora de calcular el resultado. De ser cierto, habría que omitir aquellas variables que generen ruido y que no aporten información relevante. Por eso, se va a volver a evaluar reduciendo el número de variables del conjunto de datos, tal y como se explica en el anexo 8.4. Estos son los resultados en el conjunto de validación:

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,7043	0,7043	0,7043	0,6304	0,5217	0,7087
F1-Score clase negativa	0,71	0,71	0,71	0,60	0,14	0,69
F1-Score clase positiva	0,70	0,70	0,70	0,66	0,67	0,72

Cuadro 7: Resultados sobre el conjunto de validación con las mejores variables.

En el Cuadro 7 el modelo que ha dado mejores resultados es el *SVM*, que supera por muy poco a los *Árboles de Decisión* y al *Random Forest*. Sin embargo, el resultado final depende del conjunto de prueba. Para ello, se añade al conjunto de entrenamiento el de validación, se reentrenan los modelos y se evalúan sobre el de prueba. Estos son los resultados finales:

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,7056	0,7056	0,7208	0,6429	0,6255	0,6926
F1-Score clase negativa	0,71	0,71	0,72	0,63	0,52	0,68
F1-Score clase positiva	0,70	0,70	0,72	0,65	0,69	0,70

Cuadro 8: Resultados sobre el conjunto de prueba con las mejores variables.

En el Cuadro 8 el mejor modelo (ahora sí con una diferencia notable) es el *Random Forest*, que obtiene los mejores resultados en todas las métricas. Concretamente en F-Score ha obtenido un valor de 0,72. Este es el modelo que se utilizará de aquí en adelante para procesar los datos.

5. Aproximaciones Multi Tarea

Hasta ahora se han estudiado solo los datos en forma de imagen. En esta sección se van a analizar cómo el uso de más tipos de datos afecta al rendimiento de los modelos. Es por ello por lo que se va a utilizar un modelo Multi Tarea, de forma que se extraiga más información de las imágenes y se obtenga un modelo más robusto. Estos modelos buscan resolver varias tareas relacionadas a la vez, en vez de usar un modelo para cada tarea individual. Este tipo de modelos generalizan mejor y, al aprender las características comunes a las tareas y compartir información entre esas características, mejora los resultados frente a los modelos individuales. Hay dos aproximaciones para crear un modelo Multi Tarea.

5.1. Primera aproximación

La primera aproximación es a partir de varias entradas, predecir una única salida. En este caso sería pasarle al modelo la imagen de la mamografía y extraer características de ella para pasársela como información junto al resto de datos a un modelo que devuelva el resultado final, como se puede ver en la figura 6.

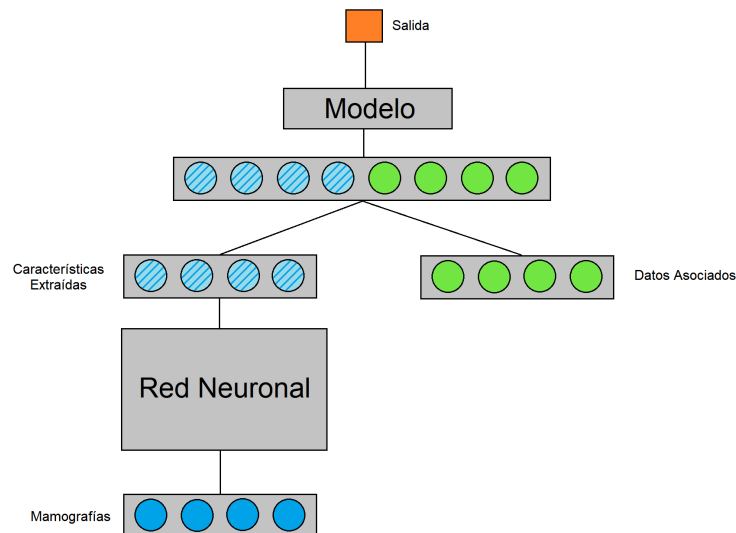


Figura 6: Aproximación 1 para el modelo Multi Tarea. A partir de varias entradas, dar una salida.

Para el modelo de extracción de características para las mamografías se van a utilizar las arquitecturas usadas en el apartado 4.1. El procedimiento es entrenar el modelo como se ha hecho en ese apartado pero, en vez de recoger la predicción sobre la variable '*cancer*', se va a recoger la salida de la penúltima capa de la Red, es decir, el vector de características de la imagen. Con la idea de pasar ese vector y los datos relacionados con las variables '*age*' y '*density*' del conjunto de datos a un modelo como los del apartado 4.2 para predecir con

toda esa información la salida, que será, ahora sí, la variable ‘*cancer*’. Con las arquitecturas vistas a cada una hay que quitarle un número determinado de capas en la última instancia del modelo.

- *ResNet 50*: se quitan las cuatro últimas capas.
- *EficientNet B3*: se quitan las cuatro últimas capas.
- *ConvNext*: se quitan las cuatro últimas capas.
- *HrNet w44*: se quitan las cuatro últimas capas.
- *DeiT3 base patch 16 384*: se quitan las dos últimas capas.

Tras hacer esto con los modelos, se extraen las características de los conjuntos de datos (80 % conjunto de entrenamiento y 20 % para el de validación) y se crean los *DataFrames* juntando las variables ‘*age*’ y ‘*density*’ con las características extraídas. Esto se hace mediante la siguiente función:

```
1 def creaDF(imagenes, caracteristicas, df):
2     dfb = df[['age', 'cancer', 'density']].copy() # Se seleccionan las
    variables objetivo
3     lista = []
4     for i in range(len(caracteristicas)): # Por cada vector de
    caracteristicas
5         img = imagenes[i]
6         serie = dfb.iloc[img]
7         car = caracteristicas[i].cpu().numpy() # Se transforma el vector
    de caracteristicas de tensor a ndarray
8         for j in range(len(car)):
9             serie = pd.concat([serie, pd.Series({j: car[j]})]) # Por cada
    caracteristica, se agrega una columna a la serie
10        lista.append(serie)
11    return pd.DataFrame(lista) # Se crea una DataFrame con todas las
    Series
```

A esta función se le pasan la lista de imágenes del conjunto en concreto junto a los vectores de características asociados a ellas. También se le pasa como argumento el *DataFrame* con los datos para poder extraer la edad y la densidad de cada imagen. Cada vector tiene 512 características. El *DataFrame* de salida se separa en *X*, con todas las variables salvo la variable objetivo, e *y*, con esa única variable. Una vez se tienen los datos estructurados, se pasan al segundo modelo. Para este segundo modelo se han usado las mismas arquitecturas que en el apartado 4.2.

<i>Resnet50</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6334	0,6052	0,5965	0,6052	0,4664	0,5900
F1-Score clase negativa	0,64	0,58	0,62	0,56	0,34	0,55
F1-Score clase positiva	0,63	0,63	0,57	0,64	0,55	0,62
<i>EficientNet B3</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6161	0,4794	0,4490	0,4946	0,5119	0,5033
F1-Score clase negativa	0,62	0,53	0,45	0,43	0,65	0,10
F1-Score clase positiva	0,61	0,42	0,44	0,55	0,20	0,66
<i>ConvNext</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6269	0,6247	0,5879	0,5987	0,5206	0,5900
F1-Score clase negativa	0,63	0,64	0,61	0,64	0,53	0,58
F1-Score clase positiva	0,62	0,61	0,57	0,54	0,51	0,60
<i>HrNet w44</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6226	0,6030	0,6074	0,5705	0,5727	0,4989
F1-Score clase negativa	0,62	0,62	0,65	0,62	0,59	0,15
F1-Score clase positiva	0,62	0,58	0,56	0,51	0,55	0,64
<i>Deit3 base patch 16 384</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6356	0,6291	0,5965	0,5835	0,5705	0,5965
F1-Score clase negativa	0,64	0,62	0,61	0,58	0,57	0,61
F1-Score clase positiva	0,63	0,64	0,58	0,59	0,57	0,58

Cuadro 9: Resultados del modelo Multi Tarea siguiendo la primera aproximación.

El el Cuadro 9 se puede ver que los mejores resultados los ha dado siempre el modelo del *Árbol C4.5*, concretamente los mejores resultados los han dado las arquitecturas *Resnet50* y *Deit3 base patch 16 384*. Sin embargo, los resultados del apartado 4.2, sin añadir las características de las imágenes se tenían mejores resultados. Para ver si es un tema de dimensión entre las características, se va a probar a normalizar el conjunto de datos y las características antes de hacer la evaluación.

<i>Resnet50</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,5965	0,5531	0,5879	0,6182	0,5662	0,5748
F1-Score clase negativa	0,59	0,57	0,59	0,61	0,60	0,57
F1-Score clase positiva	0,60	0,53	0,58	0,63	0,53	0,58
<i>EfcientNet B3</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,5553	0,5393	0,5228	0,6117	0,5119	0,6095
F1-Score clase negativa	0,56	0,55	0,62	0,64	0,57	0,53
F1-Score clase positiva	0,55	0,51	0,36	0,58	0,43	0,67
<i>ConvNext</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,5987	0,6312	0,5857	0,5987	0,5336	0,5879
F1-Score clase negativa	0,62	0,63	0,60	0,60	0,58	0,59
F1-Score clase positiva	0,57	0,63	0,57	0,60	0,48	0,59
<i>HrNet w44</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6291	0,6182	0,5662	0,5813	0,5510	0,5488
F1-Score clase negativa	0,65	0,63	0,57	0,58	0,57	0,54
F1-Score clase positiva	0,60	0,60	0,57	0,58	0,53	0,56
<i>Deit3 base patch 16 384</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,5510	0,5597	0,5835	0,5423	0,5531	0,5466
F1-Score clase negativa	0,46	0,59	0,58	0,53	0,54	0,54
F1-Score clase positiva	0,62	0,53	0,58	0,55	0,57	0,56

Cuadro 10: Resultados del modelo Multi Tarea siguiendo la primera aproximación, aplicando normalización.

Como se puede observar en el Cuadro 10, el mejor resultado lo ha tenido el *Árbol CART* extrayendo las características con el modelo *ConvNext*. Sin embargo, los resultados del Cuadro 9, son mejores en general, por lo que esta normalización no mejora los resultados.

5.2. Segunda aproximación

La segunda aproximación consiste en pasarle al modelo Multi Tarea una entrada y que devuelva varias salidas. Para este problema, esto se traduce en pasarle al modelo la mamografía y que, a partir de la imagen, se devuelvan las predicciones sobre las variables y/o clases de salida, tal y como se puede ver en la figura 7.

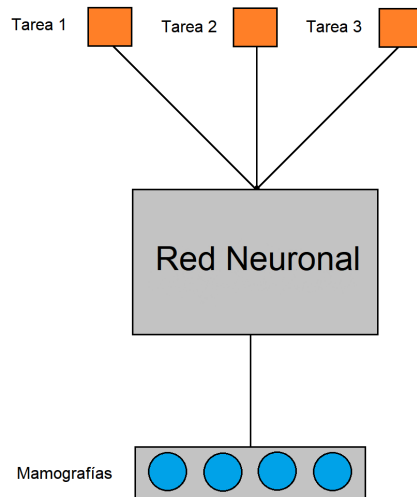


Figura 7: Aproximación 2 para el modelo Multi Tarea. A partir de una entrada, dar varias salidas.

En esta segunda aproximación, además de predecir la variable ‘*cancer*’, también se van a predecir las variables ‘*age*’ y ‘*density*’. La elección de estas variables es bastante obvia, pues son las dos variables que más influyen en el resultado final, pues una densidad elevada puede ser indicativo de cáncer y dificulta su detección, mientras que la edad también influye, pues las probabilidades de padecer la enfermedad aumenta con los años. También, como se ha visto en los modelos de datos, solamente con esas dos variables se pueden hacer predicciones con buenos resultados, de hecho, las mejores predicciones de entre todas las combinaciones probadas.

Escoger estas variables a priori parece un reto, pues cada una es de un tipo distinto, la variable ‘*age*’ es una variable numérica, la variable ‘*cancer*’ es una variable binaria y la variable ‘*density*’ es una variable categórica. Es por eso por lo que cada una tendrá una métrica distinta y una función de pérdida propia para cada una, que a su vez influirán de manera distinta en la función de pérdida general del modelo.

5.2.1. Descripción de la arquitectura Multi Tarea de la segunda aproximación

Para realizar esta tarea, se ha basado la arquitectura en la desarrollada por T. Dantas en *Medium*[7], donde se desarrolla un modelo de características similares en una versión de *FastAI*.

El primer paso es cómo se van a pasar los datos al modelo, para que éste aprenda de ellos. Para ello se crea una clase que se usará para cumplir dicha función. Ésta se crea pasándole un *DataFrame*, que funcionará como base del conjunto de datos, y una lista de transformaciones que se le aplicarán. Al inicializar la clase, se recoge la información de cada

imagen y cada paciente para guardar las direcciones de las imágenes y poder leerlas más tarde. También se recogen las variables objetivo (que en un primer momento se han juntado en la variable *'label'*) para poder entrenar al modelo y evaluarlo. Finalmente, se aplican unas transformaciones de normalización siguiendo las estadísticas de *ImageNet*. También se define adicionalmente el contador de tamaño, el selector de objeto (que devuelve la imagen como tensor, y los valores de las variables objetivo y para ello se lee la imagen y le aplica las transformaciones) y la función de muestra (que enseña los datos para comprobar los resultados). Como apunte, para la edad se calcula su logaritmo neperiano y se divide por 4,489. El logaritmo neperiano sirve para no dar más importancia a las edades más altas y tomar todos los fallos por igual, y la división por 4,489 se debe a que ese es el máximo valor del logaritmo de la edad (que son 89 años) y de esta manera el valor es un número de entre 0 y 1.

```

1 class MultiTaskDataset():
2     def __init__(self, df, tfms, size=64):
3         self.paths = []
4         for index, row in df.iterrows(): # Se lee el DataFrame
5             patient = row["patient_id"]
6             image = row["image_id"]
7             path = p_under2/str(patient)/(str(image) + '.jpg')
8             self.paths.append(path)
9         self.labels = list(df["label"])
10        self.tfms = tfms
11        self.size = size
12        self.train = df.copy()
13        self.norm = transforms.Normalize([0.485, 0.456, 0.406], [0.229,
0.224, 0.225])
14
15        def __len__(self): return len(self.paths)
16
17        def __getitem__(self, idx):
18            # Se lee la imagen y se transforma
19            img = PIL.Image.open(self.paths[idx]).convert('RGB')
20            img = img.resize((256, 256))
21            t = torch.Tensor(np.array(img))
22            t = t.permute(2, 0, 1).float() / 255.0
23            transform = transforms.ToPILImage()
24            img = PILImage(PILImage.create(transform(t)))
25            img = self.tfms.encode(img)
26            transform = transforms.Compose([transforms.ToTensor()])
27            img = self.norm(transform(img))
28            # Se tratan las variables
29            labels = self.labels[idx].split(" ")
30            age = torch.tensor(float(labels[0]), dtype=torch.float32)
31            cancer = torch.tensor(int(labels[1]), dtype=torch.int64)
32            density = torch.tensor(int(labels[2]), dtype=torch.int64)
33
34            return img, (age.log_() / 4.489, cancer, density)
35
36        def show(self, idx):
37            x, y = self.__getitem__(idx)
38            age, cancer, density = y
39            stds = np.array([0.229, 0.224, 0.225])

```

```

40     means = np.array([0.485, 0.456, 0.406])
41     img = ((x.numpy().transpose((1, 2, 0)) * stds + means) * 255).
    astype(np.uint8)
42     plt.imshow(img)
43     plt.title("{} {} {}".format(int(age.mul_(4.489).exp_().item()),
    cancer.item(), density.item()))

```

Para el modelo Multi Tarea, también se crea una clase. Se inicializa siguiendo la misma estructura de la figura 7, con un cuerpo y tres cabezas. El cuerpo puede inicializarse siguiendo cualquier arquitectura de las de *FastAI* y su función es la de extraer características para pasárselas a las tres cabezas, una para cada tarea. A cada tarea se le pasa también el posible número de salidas que puede dar. Para la cabeza de la edad, se le ha aplicado una sigmoide a su resultado para forzar al modelo a que la salida no se salga del rango de edades, es decir, que no de resultados como por ejemplo puedan ser 116 años, y que como mucho la edad máxima que devuelva sea de 89 años. La respuesta final del modelo es una lista con los resultados de cada cabeza.

```

1 class MultiTaskModel(nn.Module):
2     def __init__(self, arch, ps=0.5):
3         super(MultiTaskModel, self).__init__()
4         self.encoder = create_body(arch)      # Crea un encoder dada una
    arquitectura de FastAI
5         self.fc1 = create_head(1024, 1, ps=ps)
6         self.fc2 = create_head(1024, 2, ps=ps)
7         self.fc3 = create_head(1024, 4, ps=ps)
8
9     def forward(self, x):
10        x = self.encoder(x)
11        age = torch.sigmoid(self.fc1(x))
12        cancer = self.fc2(x)
13        density = self.fc3(x)
14
15        return [age, cancer, density]

```

En cuanto a la métrica, para la variable ‘age’ se utilizará la raíz del error cuadrático medio, que para tareas de regresión funciona muy bien. Para la variable ‘cancer’ se mantiene el F1-Score, por las mismas razones comentadas con anterioridad. Y para la variable ‘density’ la métrica de Accuracy. Para las funciones de pérdida, se usará *MSELossFlat* en la variable numérica y *CrossEntropyLossFlat* en las binarias y categóricas. El valor óptimo de la función *MSELossFlat* es 0, mientras que para *CrossEntropyLossFlat* ese valor es 1.

```

1 class MultiTaskLossWrapper(nn.Module):
2     def __init__(self, task_num):
3         super(MultiTaskLossWrapper, self).__init__()
4         self.task_num = task_num
5         self.log_vars = nn.Parameter(torch.zeros((task_num)))
6
7     def forward(self, preds, label):
8         mse, crossEntropy = MSELossFlat(), CrossEntropyLossFlat() #
    Funciones de perdida
9
10        loss0 = mse(preds[0], label[0])

```

```

11     loss1 = crossEntropy(preds[1], label[1])
12     loss2 = crossEntropy(preds[2], label[2])
13
14     precision0 = torch.exp(-self.log_vars[0])
15     loss0 = precision0 * loss0 + self.log_vars[0]
16
17     precision1 = torch.exp(-self.log_vars[1])
18     loss1 = precision1 * loss1 + self.log_vars[1]
19
20     precision2 = torch.exp(-self.log_vars[2])
21     loss2 = precision2 * loss2 + self.log_vars[2]
22
23     return loss0 + loss1 + loss2 # Valor de perdida final

```

En primera instancia todas las funciones tienen el mismo peso y se aplicará el callback de *Early Stopping* sobre el F1-Score de la variable ‘*cancer*’, que es la que más interesa predecir bien.

5.2.2. Resultados de la arquitectura Multi Tarea de la segunda aproximación

Como arquitecturas que se pueden pasar al modelo Multi Tarea no se han tenido en cuenta las de *HrNet w44* y *Deit3 base patch 16 384*, por ser arquitecturas de por sí pesadas y realizar varias tareas sobre un modelo así lo ralentiza aún más. Los resultados de esta arquitectura Multi Tarea con los distintos modelos se obtiene la siguiente tabla:

	<i>Resnet50</i>	<i>EfficientNet</i>	<i>ConvNext</i>
rmse_age	0,0486	0,0475	0,0564
f1_cancer	0,5540	0,6092	0,6179
acc_density	0,6052	0,5813	0,3471
Tiempo (Minutos)	3	3	13
Época	3	3	5

Cuadro 11: Resultados sobre el conjunto de imágenes balanceado por imágenes sobre una arquitectura Multi Tarea.

En el Cuadro 11 se puede ver que el modelo no tiene ningún problema en predecir la edad, con cualquiera de las arquitecturas probadas, respecto al cáncer se obtiene un F1-Score parecido a los modelos individuales, pero ligeramente inferiores y, en relación a la densidad, el Accuracy no es muy alto. El modelo con mejor valor en la métrica de F1-Score sobre la variable ‘*cancer*’ ha sido *ConvNext*, seguido muy de cerca por *EfficientNet*.

Resulta que al tener todas las funciones de pérdida con el mismo peso, el modelo intenta ajustar todas por igual, porque les da a todas la misma importancia. Esto lo que provoca es que el modelo ve como una mejora el conseguir ajustar mejor una variable aún perdiendo ajuste en otras si la mejora de la variable concreta compensa el empeoramiento. Por ejemplo, si en la variable ‘*age*’ se mejora bastante a costa de empeorar en menor medida los resultados

sobre la variable ‘*cancer*’, el modelo va a intentar seguir ese camino pese a lo que realmente se busca es una mejora en la variable ‘*cancer*’. Viendo estos resultados, se va a ajustar la función de pérdida para intentar mejorar la predicción de la variable ‘*cancer*’, asignando pesos de 0,6 al ajuste del cáncer, 0,3 para la densidad y 0,1 para la edad, que es la que mejor calcula.

```
1 return 3 * (0.1 * loss0 + 0.6 * loss1 + 0.3 * loss2) # Valor de
   perdida final
```

Con estos cambios, los resultados finales son:

	<i>Resnet50</i>	<i>EfficientNet</i>	<i>ConvNext</i>
rmse_age	0,0396	0,0411	0,0377
f1_cancer	0,6150	0,5551	0,5812
acc_density	0,5879	0,5510	0,6052
Tiempo (Minutos)	4	1	2
Época	6	1	1

Cuadro 12: Resultados sobre el conjunto de imágenes balanceado por imágenes sobre una arquitectura Multi Tarea ajustando la función de pérdida.

Con estos resultados del Cuadro 12, los mejores resultados los da de nuevo el modelo *Resnet50*. Si se comparan los resultados con los vistos en el Cuadro 11, el modelo *ConvNext* tiene un F1-Score ligeramente superior. El ajuste de la función de pérdida ha empeorado los resultados en todos los modelos salvo en *Resnet50*.

6. Conclusiones

7. Bibliografía y Referencias

Referencias

- [1] *Cáncer de mama*. Visto el 23-06-2023. 2023. URL: https://es.wikipedia.org/wiki/C%C3%A1ncer_de_mama.
- [2] *Cáncer de mama: Estadísticas*. Visto el 23-06-2023. 2022. URL: <https://www.cancer.net/es/tipos-de-c%C3%A1ncer/c%C3%A1ncer-de-mama/estad%C3%ADsticas>.
- [3] *FastAI*. 2016. URL: <https://www.fast.ai/>.
- [4] Kaiming He et al. «Deep Residual Learning for Image Recognition». En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Jun. de 2016.
- [5] Brett Koonce y Brett Koonce. «EfficientNet». En: *Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization* (2021), págs. 109-123.
- [6] Zhuang Liu et al. «A ConvNet for the 2020s». En: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Jun. de 2022, págs. 11976-11986.
- [7] *Multi-Task Learning with Pytorch and FastAI*. Visto el 28-06-2023. 2020. URL: <https://towardsdatascience.com/multi-task-learning-with-pytorch-and-fastai-6d10dc7ce855>.
- [8] Radiological Society of North American. *RSNA Screening Mammography Breast Cancer Detection*. 2022. URL: <https://kaggle.com/competitions/rsna-breast-cancer-detection>.
- [9] *NumPy*. 1995. URL: <https://numpy.org/>.
- [10] Sociedad Española de Oncología Médica (SEOM). *Las cifras del cáncer en España 2020*. Página 6.
- [11] *Pandas*. 2022. URL: <https://pandas.pydata.org/>.
- [12] *PyTorch*. 2017. URL: <https://pytorch.org/>.
- [13] *scikit-learn*. 2010. URL: <https://scikit-learn.org/>.
- [14] *timm*. 2022. URL: <https://timm.fast.ai/>.
- [15] *Torch*. 2002. URL: <http://torch.ch/>.
- [16] Hugo Touvron et al. «Training data-efficient image transformers & distillation through attention». En: *International conference on machine learning*. PMLR. 2021, págs. 10347-10357.
- [17] Jingdong Wang et al. «Deep high-resolution representation learning for visual recognition». En: *IEEE transactions on pattern analysis and machine intelligence* 43.10 (2020), págs. 3349-3364.

8. Anexos

8.1. Descripción del Conjunto de Datos

El conjunto de datos representa la información de las imágenes mediante una serie de variables o atributos, siendo éstas y su distribución la siguiente:

- **'site_id'**: El identificador del hospital de origen, en el que se realizaron las imágenes y se recogieron los datos. Hay 2 hospitales distintos desde los que provienen las imágenes.
 - Hospital 1: 29.519 imágenes (54 %).
 - Hospital 2: 25.187 imágenes (46 %).
- **'patient_id'**: El identificador del paciente. En total hay 11.913 pacientes distintos, cada uno con sus imágenes.
- **'image_id'**: El identificador de la imagen. Cada fila del conjunto de datos representa a una imagen. Hay 54.706 imágenes en total, y no hay ninguna repetida.
- **'laterality'**: Indica si la imagen se ha tomado sobre el pecho izquierdo (L) o sobre el derecho (R). Hay casi las mismas imágenes para cada lado.
 - Lado derecho (R): 27.439 imágenes (50 %).
 - Lado izquierdo (L): 27.267 imágenes (50 %).
- **'view'**: Indica la orientación de la imagen. El procedimiento por defecto es capturar dos vistas por cada pecho. Hay dos vistas muy usadas, con números muy parecidos, y otras vistas muy poco usadas. Esto es normal, pues las vistas *MLO* y *CC* son las vistas estándar.
 - *MLO* (Oblicua Mediolateral): 27.903 imágenes (52 %).
 - *CC* (Craneocaudal): 26.765 imágenes (48 %).
 - *AT* (Cola Axilar): 19 imágenes (0 %).
 - *LM* (Lateromedial): 10 imágenes (0 %).
 - *ML* (Mediolateral): 8 imágenes (0 %).
 - *LMO* (Oblicua Lateromedial): 1 imagen (0 %).
- **'age'**: Indica la edad del paciente en años. Distribución con forma normal centrada en los 58 años. Las edades varían desde los 26 años hasta los 89.
- **'implant'**: Indica si el paciente tiene implantes de pecho. El hospital con el identificador 1 da la información de los implantes a nivel de paciente, no de pecho, por lo que en este caso no se puede determinar con certeza que el implante sea un implante de pecho. La clase sin implantes tiene 53.229 imágenes (97 %) y la clase con implantes tiene 1.477 imágenes (el 3 % restante).
- **'density'**: Es una puntuación que indica lo denso que es el tejido del pecho, asignando A para la menor densidad y D para la mayor. Se indica que un tejido más denso puede dificultar el diagnóstico. Esta variable sólo aparece en el conjunto de Entrenamiento. Hay 4 clases que siguen una distribución normal:

- *NaN*: Hay bastantes entradas sin valor, concretamente 25.236, lo que representa un 46 % del total.
 - *A*: 3.105 entradas (6 % del total).
 - *B*: 12.651 entradas (23 %).
 - *C*: 12.175 entradas (22 %).
 - *D*: 1.539 entradas (3 %).
- **‘machine_id’**: Un identificador para la máquina que sacó la imagen. Se han usado 10 máquinas para realizar las mamografías.
- Máquina 21: 8.221 imágenes (15 %).
 - Máquina 29: 8.267 imágenes (15 %).
 - Máquina 48: 8.699 imágenes (16 %).
 - Máquina 49: 23.529 imágenes (43 %).
 - Máquina 93: 1.915 imágenes (3,35 %).
 - Máquina 170: 923 imágenes (2 %).
 - Máquina 190: 145 imágenes (0,3 %).
 - Máquina 197: 29 imágenes (0,0005 %).
 - Máquina 210: 1.070 imágenes (2 %).
 - Máquina 216: 1.908 imágenes (3,35 %).
- **‘cancer’**: Es la variable objetivo, indica si el pecho ha dado positivo en cáncer o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene cáncer), tiene 53.548 imágenes (98 %) y la clase positiva tiene 1.158 imágenes (2 %).
- **‘biopsy’**: Indica si una biopsia se ha realizado en el pecho. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene biopsia) tiene 51.737 imágenes (94,5 %) y la clase positiva tiene 2.969 imágenes (5,5 %).
- **‘invasive’**: En caso de detectar cáncer, indica si el mismo es invasivo o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (sin cáncer invasivo) tiene 53.888 imágenes (98,5 %) y la clase positiva tiene 818 imágenes (1,5 %).
- **‘BIRADS’**: 3 clases.
- *NaN*: 28.420 imágenes sin valor asociado (52 %).
 - *0*: El pecho necesita seguimiento en 8.249 imágenes (15 %).
 - *1*: El pecho ha dado negativo en cáncer en 15.772 imágenes (29 %).
 - *2*: El pecho es normal en 2.265 imágenes (4 %).
- **‘prediction_id’**: El identificador para la fila que se corresponde con la predicción. Esta variable sólo aparece en el conjunto de Test.

- *‘difficult_negative_case’*: Tiene valor positivo si el caso fue más difícil de lo habitual. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa tiene 47.001 entradas (86 %) y la positiva 7.705 (14 %).

Es curioso que haya más imágenes con biopsia que con cáncer, entonces es de suponer que la biopsia se ha realizado antes que la imagen, y puede ser que sean pacientes que hayan tenido cáncer de mama en el pasado, pero que actualmente no lo tienen.

8.2. Resultados sobre los distintos conjuntos

En este apartado se muestran los resultados obtenidos con el resto de conjuntos de datos, siendo estos, el conjunto sin balancear y el balanceado por pacientes, con el objetivo de poder compararlos con los resultados finales.

La primera evaluación se ha realizado con los modelos utilizando los parámetros por defecto y un tamaño de imagen de 256x256 (salvo con *Deit*, ya que el modelo exige una resolución de 384x384). El valor del ratio de aprendizaje es de 1e-2 y se ejecuta durante 10 épocas. El conjunto de datos es el original sin balancear, de 54.706 instancias.

	<i>Resnet50</i>	<i>EfficientNet</i>	<i>ConvNext</i>	<i>Hrnet</i>
Accuracy	0,9773	0,9804	0,9805	0,9765
Precision	0,4375	0,0000	0,5385	0,6250
Recall	0,0574	0,0000	0,0327	0,0193
F-Score	0,1015	0,0000	0,0617	0,0375
Tiempo (Minutos)	81	9	66	98
Época	9	1	8	8

Cuadro 13: Resultados sobre todo el conjunto de imágenes.

En el Cuadro 13 el modelo *Deit*, al usar las imágenes más grandes y ser un modelo más pesado, no se evalúa pues sólo en ejecutar la primera época tarda una hora. Se puede ver que las épocas no son rápidas, tardan más o menos 10 minutos cada una y esto es debido al gran volumen de imágenes que hay. También, los altos valores de Accuracy indican que las predicciones están siendo de la clase negativa, sin detectar las instancias con cáncer, por lo que no está prediciendo realmente si tiene cáncer o no.

Al evaluar sobre el balanceo por pacientes, se tiene un volumen de datos muy inferior al anterior, por lo que los tiempos son bastante inferiores y por tanto, se pueden entrenar más épocas. Se entrena durante 30 épocas y se reduce el ratio de aprendizaje a 1e-3. Concretamente, a *Resnet50*, por haber sido la que mejores resultados ha tenido con el conjunto anterior, se le ha aumentado el número de épocas respecto a los demás para ver si puede alcanzar resultados superiores, aunque esto pueda llevar al sobreajuste.

	<i>Resnet50</i>	<i>ConvNext</i>	<i>Hrnet</i>	<i>Deit</i>
Accuracy	0,7173	0,7620	0,7009	0,7806
Precision	0,4565	0,6016	0,3878	0,5506
Recall	0,5598	0,3156	0,4744	0,4475
F-Score	0,5029	0,4140	0,4268	0,4937
Tiempo (Minutos)	57	40	80	45
Época	38	11	14	20

Cuadro 14: Resultados sobre el conjunto de imágenes balanceado por pacientes.

EfficientNet ha dado error al probar este balanceo no se sabe bien por qué. En general, como se puede ver en el Cuadro 14, los resultados son mucho mejores que en la anterior tabla. *Resnet50* ha vuelto a ser el modelo con mejor F-Score, aunque lo ha sido en una época muy avanzada. El modelo más cercano ha sido *Deit* en casi la mitad de épocas.

Aplicando el callback de *Early Stopping*, se obtienen los siguientes resultados:

	<i>Resnet50</i>	<i>EfficientNet</i>	<i>ConvNext</i>	<i>Hrnet</i>	<i>Deit</i>
Accuracy	0,6758	0,7576	0,7304	0,7413	0,8024
Precision	0,2822	0,3333	0,3603	0,3372	0,6000
Recall	0,2727	0,0622	0,2345	0,1388	0,4019
F-Score	0,2774	0,1048	0,2841	0,1966	0,4814
Tiempo (Minutos)	1	1	12	6	20
Época	1	1	2	1	9

Cuadro 15: Resultados sobre el conjunto de imágenes balanceado por pacientes, aplicando *Early Stopping*.

En el Cuadro 15 se ve que el mejor modelo ha sido *Deit*, pero tiene unos resultados inferiores a los resultados del Cuadro 14.

Ninguno de estos conjuntos obtiene unos resultados que se acerquen a los obtenidos con el conjunto balanceado por imágenes.

8.3. Ajuste de parámetros para los modelos de datos

Para el ajuste por parámetros, en las pruebas se ha utilizado un *param grid*, de forma que se prueban combinaciones entre los valores de los parámetros para buscar el modelo que de el mejor resultado. Para todos los modelos, ya que tienen una parte estocástica, se ha asignado un valor de semilla aleatoria de 0. Para los modelos, estos parámetros son los siguientes:

- **Árbol C4.5.**
 - *criterion* = ‘entropy’. (Valor por defecto del Árbol C4.5)
 - *max_depth* = 5, 10, None.
 - *min_samples_split* = 2, 10, 20.
- **Árbol CART.**

- *criterion* = ‘gini’. (Valor por defecto del *Árbol CART*)
- *max_depth* = 5, 10, None.
- *min_samples_split* = 2, 10, 20.
- ***Random Forest.***
 - *criterion* = ‘gini’, ‘entropy’.
 - *max_features* = ‘sqrt’, ‘log2’, None.
 - *bootstrap* = True, False.
- ***Regresión Logística.***
 - *C* = 0,001, 0,01, 0,1, 1.
- ***Red Neuronal.***
 - *hidden_layer_sizes* = 10, 25, 50, 100.
 - *alpha* = 0,0001, 0,001, 0,01, 0,1.
- ***SVM.***
 - *gamma* = ‘auto’ (Valor por defecto).
 - *decision_function_shape* = ‘ovo’, ‘ovr’.

Los parámetros en cada una de las pruebas se consiguen mediante la evaluación sobre el conjunto de validación. Para el conjunto de datos final, estos son los valores de los parámetros:

- ***Árbol C4.5.***
 - *criterion* = ‘entropy’. (Valor por defecto del *Árbol C4.5*)
 - *max_depth* = None.
 - *min_samples_split* = 2.
- ***Árbol CART.***
 - *criterion* = ‘gini’. (Valor por defecto del *Árbol CART*)
 - *max_depth* = None.
 - *min_samples_split* = 2.
- ***Random Forest.***
 - *criterion* = ‘entropy’.
 - *max_features* = None.
 - *bootstrap* = True.
- ***Regresión Logística.***
 - *C* = 0,01.
- ***Red Neuronal.***
 - *hidden_layer_sizes* = 50.
 - *alpha* = 0,1.
- ***SVM.***
 - *gamma* = ‘auto’ (Valor por defecto).
 - *decision_function_shape* = ‘ovo’.

8.4. Combinación Óptima de variables en el conjunto de datos

Con el objetivo de optimizar el resultado de la búsqueda del mejor conjunto de datos para el modelo, se ha creado una función para calcular todas las posibles combinaciones de variables del conjunto. En las posibles combinaciones está el conjunto vacío, por razones obvias, este conjunto no será utilizado para evaluar.

Es también importante mencionar, que los identificadores no van a ser tomados como variables a la hora de entrenar, ya que, aunque hacer esto mejora bastante los resultados, no tiene sentido, pues no tiene por qué seguir un mismo patrón a la hora de añadir nuevos datos, o lo que es lo mismo, pone en grave peligro la generalización del modelo. De esta manera se evita el sobreajuste.

Antes de realizar ninguna combinación, se evalúan los resultados de los modelos sobre este conjunto sin identificadores.

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Accuracy	0,6609	0,6565	0,6478	0,6304	0,6304	0,6304
F1-Score clase negativa	0,67	0,66	0,64	0,59	0,32	0,63
F1-Score clase positiva	0,65	0,65	0,65	0,66	0,62	0,63

Cuadro 16: Resultados sobre el conjunto de datos sin las variables con identificadores.

En el Cuadro 16 el modelo que mejor resultado ha dado ha sido el *Árbol C4.5*, por lo que as combinaciones y sus resultados se calcularán utilizando este modelo como base:

Variables	Accuracy	F1-Score clase neg	F1-Score clase pos
'laterality'	0,4805	0,48	0,48
'view'	0,4957	0,00	0,66
'age'	0,6126	0,62	0,61
'implant'	0,5043	0,05	0,66
'density'	0,5195	0,24	0,65
'laterality', 'view'	0,4675	0,28	0,58
'laterality', 'age'	0,6039	0,61	0,60
'laterality', 'implant'	0,4847	0,49	0,48
'laterality', 'density'	0,5130	0,47	0,55
'view', 'age'	0,5909	0,60	0,58
'view', 'implant'	0,5022	0,03	0,67
'view', 'density'	0,4957	0,34	0,59
'age', 'implant'	0,6190	0,63	0,61
'age', 'density'	0,7100	0,71	0,71
'implant', 'density'	0,5238	0,25	0,65
'laterality', 'view', 'age'	0,6061	0,63	0,58
'laterality', 'view', 'implant'	0,4847	0,49	0,48
'laterality', 'view', 'density'	0,4935	0,48	0,50
'laterality', 'age', 'implant'	0,6103	0,60	0,62
'laterality', 'age', 'density'	0,6494	0,66	0,63
'laterality', 'implant', 'density'	0,5173	0,48	0,55
'view', 'age', 'implant'	0,5952	0,60	0,59
'view', 'age', 'density'	0,6990	0,70	0,69
'view', 'implant', 'density'	0,5411	0,50	0,58
'age', 'implant', 'density'	0,7100	0,71	0,71
'laterality', 'view', 'age', 'implant'	0,6126	0,62	0,60
'laterality', 'view', 'age', 'density'	0,6429	0,65	0,64
'laterality', 'view', 'implant', 'density'	0,5022	0,49	0,52
'laterality', 'age', 'implant', 'density'	0,6537	0,67	0,64
'view', 'age', 'implant', 'density'	0,6926	0,70	0,68
'laterality', 'view', 'age', 'implant', 'density'	0,6429	0,65	0,63

Cuadro 15: Resultados sobre el conjunto de datos sin las variables con identificadores y las posibles combinaciones de variables.

En el Cuadro 15 se ven algunos resultados interesantes, al estar sin normalizar, la variable 'age' tiene más peso que las demás a la hora de emitir un resultado. Si se normalizan las variables para dar a cada una de ellas el mismo grado de importancia, los resultados son los siguientes:

Variables	Accuracy	F1-Score clase neg	F1-Score clase pos
'laterality'	0,4805	0,48	0,48
'view'	0,4957	0,00	0,66
'age'	0,5779	0,60	0,56
'implant'	0,5043	0,05	0,66
'density'	0,5195	0,24	0,65
'laterality', 'view'	0,4675	0,28	0,58
'laterality', 'age'	0,5736	0,59	0,56
'laterality', 'implant'	0,4847	0,49	0,48
'laterality', 'density'	0,5130	0,47	0,55
'view', 'age'	0,5736	0,60	0,55
'view', 'implant'	0,5022	0,03	0,67
'view', 'density'	0,4957	0,34	0,59
'age', 'implant'	0,5822	0,61	0,55
'age', 'density'	0,5952	0,61	0,58
'implant', 'density'	0,5238	0,25	0,65
'laterality', 'view', 'age'	0,5671	0,59	0,55
'laterality', 'view', 'implant'	0,4847	0,49	0,48
'laterality', 'view', 'density'	0,4935	0,48	0,50
'laterality', 'age', 'implant'	0,5844	0,58	0,59
'laterality', 'age', 'density'	0,5606	0,61	0,50
'laterality', 'implant', 'density'	0,5173	0,48	0,55
'view', 'age', 'implant'	0,5758	0,60	0,55
'view', 'age', 'density'	0,5974	0,61	0,58
'view', 'implant', 'density'	0,5411	0,50	0,58
'age', 'implant', 'density'	0,5952	0,61	0,58
'laterality', 'view', 'age', 'implant'	0,5649	0,57	0,56
'laterality', 'view', 'age', 'density'	0,5693	0,60	0,53
'laterality', 'view', 'implant', 'density'	0,5022	0,49	0,52
'laterality', 'age', 'implant', 'density'	0,5693	0,61	0,52
'view', 'age', 'implant', 'density'	0,5974	0,61	0,58
'laterality', 'view', 'age', 'implant', 'density'	0,5736	0,60	0,54

Cuadro 16: Resultados sobre el conjunto de datos normalizado sin las variables con identificadores y las posibles combinaciones de variables.

En el Cuadro 16 se puede observar que el rendimiento baja respecto al conjunto sin normalizar (Cuadro 15), ya que el valor de la edad, que es la característica más importante del conjunto, y al normalizarlo, pierde fuerza en el resultado final. Si se comparan las filas normalizadas y sin normalizar, se puede ver que aquellas sin la variable 'age' no presentan cambios, mientras que las que sí, pierden en rendimiento.

Como se puede ver, los dos mejores conjuntos son estos:

- *X1* (Accuracy: 0,7100, F1-Score en clase positiva 0,71 y F1-Score en clase negativa 0,71)
 - ‘*age*’.
 - ‘*density*’.
- *X2* (Accuracy: 0,7100, F1-Score en clase positiva 0,71 y F1-Score en clase negativa 0,71)
 - ‘*age*’.
 - ‘*implant*’.
 - ‘*density*’.

El segundo con una variable más obtiene los mismos resultados. Sin embargo, como se menciona en la descripción del conjunto de datos, los hospitales computan de manera distinta esta variable, pues el hospital 1 apuntaba cualquier tipo de implante, y el hospital 2 apuntaba sólo los implantes en los pechos.

Por esta razón y ya que la diferencia de rendimiento entre ambos es inexistente, el mejor conjunto de datos para el modelo es el que cuenta únicamente con las variables ‘*age*’ y ‘*density*’.

8.5. Modelo Multi Tarea, Aproximación 2 con función de pérdida única

Viendo los resultados del apartado 5.2.2, se va a probar a cambiar la función de pérdida del modelo Multi Tarea teniendo en cuenta únicamente la pérdida asociada a la variable ‘*cancer*’, para ver cómo afecta al resultado del modelo. Esto teóricamente no tendría mucho sentido, pues al hacer esto, el modelo sólo se fijaría en las características relacionadas con esa variable, obviando aquellas de las otras dos variables y haciendo que el Multi Tarea pierda su sentido. Sin embargo, en la práctica puede que esto sea diferente. Este es el resultado:

	<i>Resnet50</i>	<i>EfficientNet</i>	<i>ConvNext</i>
rmse_age	0,3090	0,3419	0,5070
f1_cancer	0,5925	0,4625	0,6385
acc_density	0,0889	0,2560	0,1193
Tiempo (Minutos)	7	1	7
Época	9	1	3

Cuadro 17: Resultados sobre el conjunto de imágenes balanceado por imágenes sobre una arquitectura Multi Tarea teniendo en cuenta únicamente la variable ‘*cancer*’ para la función de pérdida.

Se puede ver en el Cuadro 17 que los resultados se corresponden con la teoría, y los resultados son peores que los del modelo Multi Tarea con la función de pérdida ajustada. Es notable especialmente en las variables '*age*' y '*density*' el empeoramiento de los resultados. Como teóricamente se ha dicho, al no tener tanto contexto y no fijarse en la información de las otras dos variables, da unos resultados similares a los del modelo individual. Es por esto por lo que una función de pérdida teniendo en cuenta solamente la variable '*cancer*' no tiene sentido.